Specification Seems Hard

Mike Dodds - Oath Technologies

2026-11-07



Oath is what

A Focused Research Organization (FRO)

Working on *formal oversight*

- AI may be very dangerous
- AIs will do things bigger / faster / subtler than we can oversee
- Can we scale our oversight / trust using formal verification?

Based in Berkeley, CA

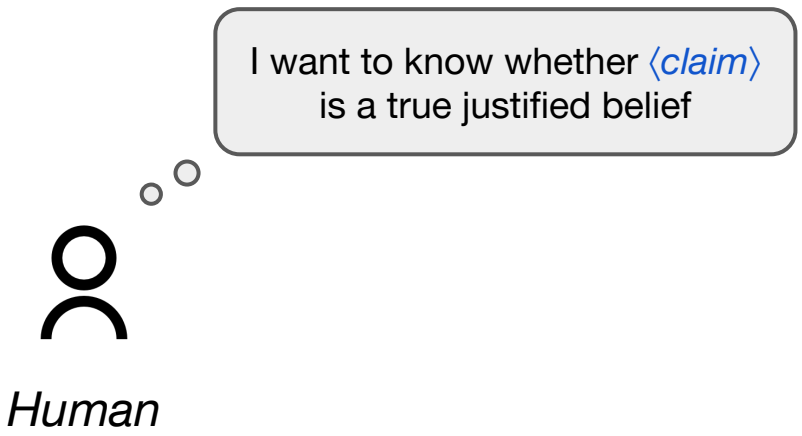
Hiring: https://oath.tech/jobs/

Oath

1. Why formally verify things anyway
2. Specification seems hard
3. What can we do



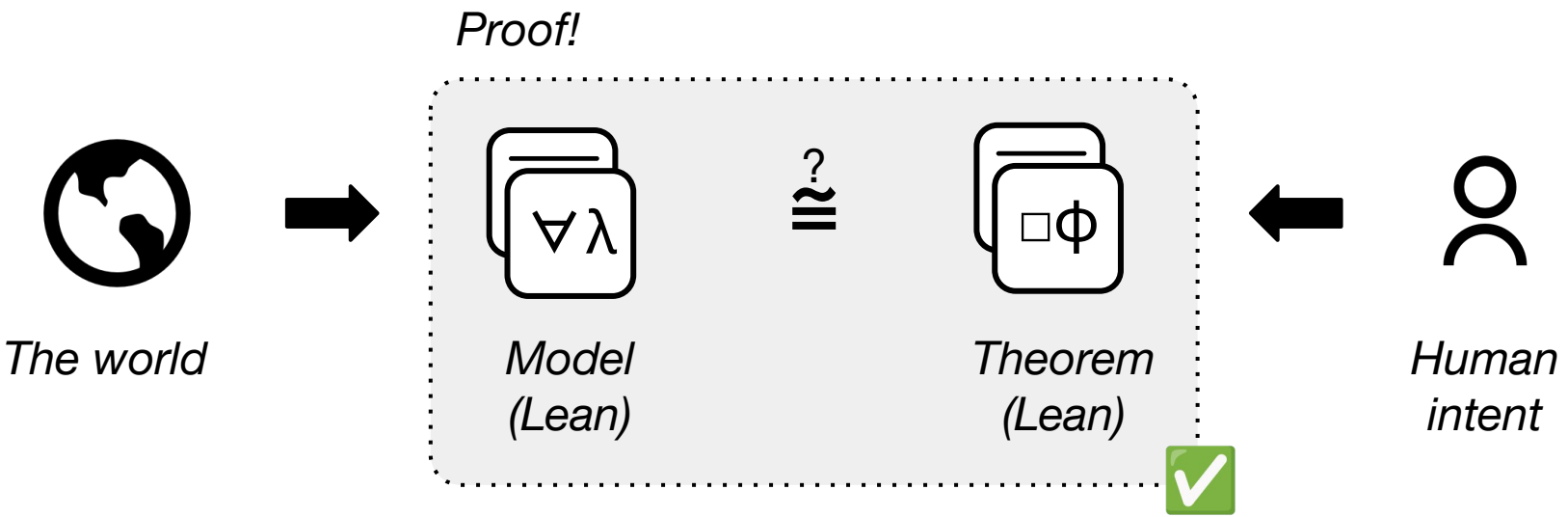
Why formally verify things?



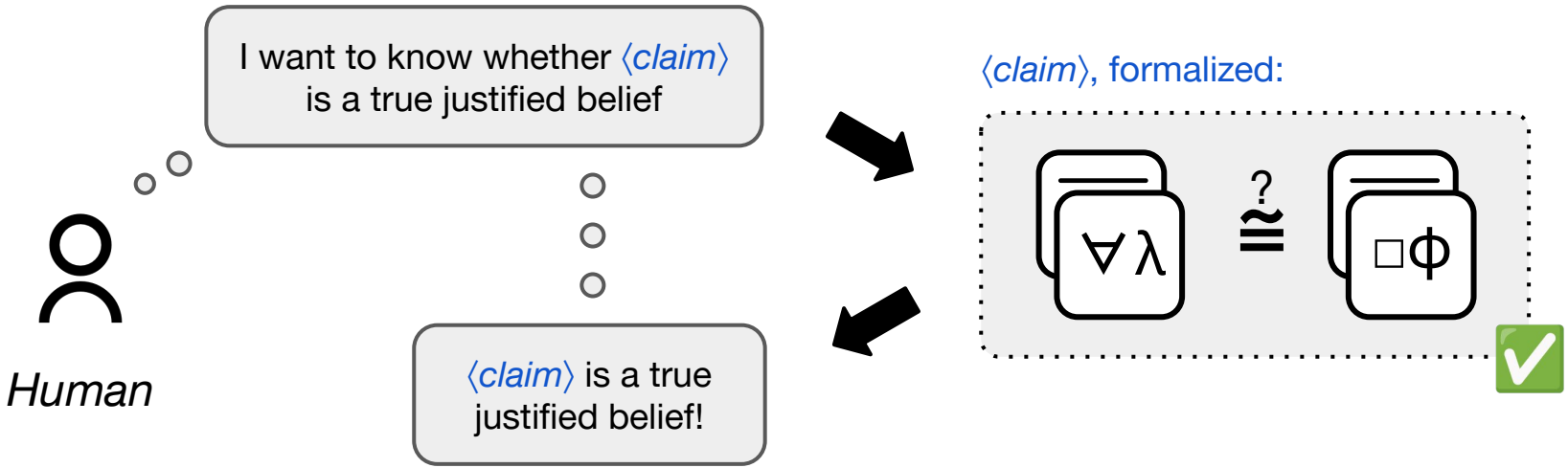
Eg:

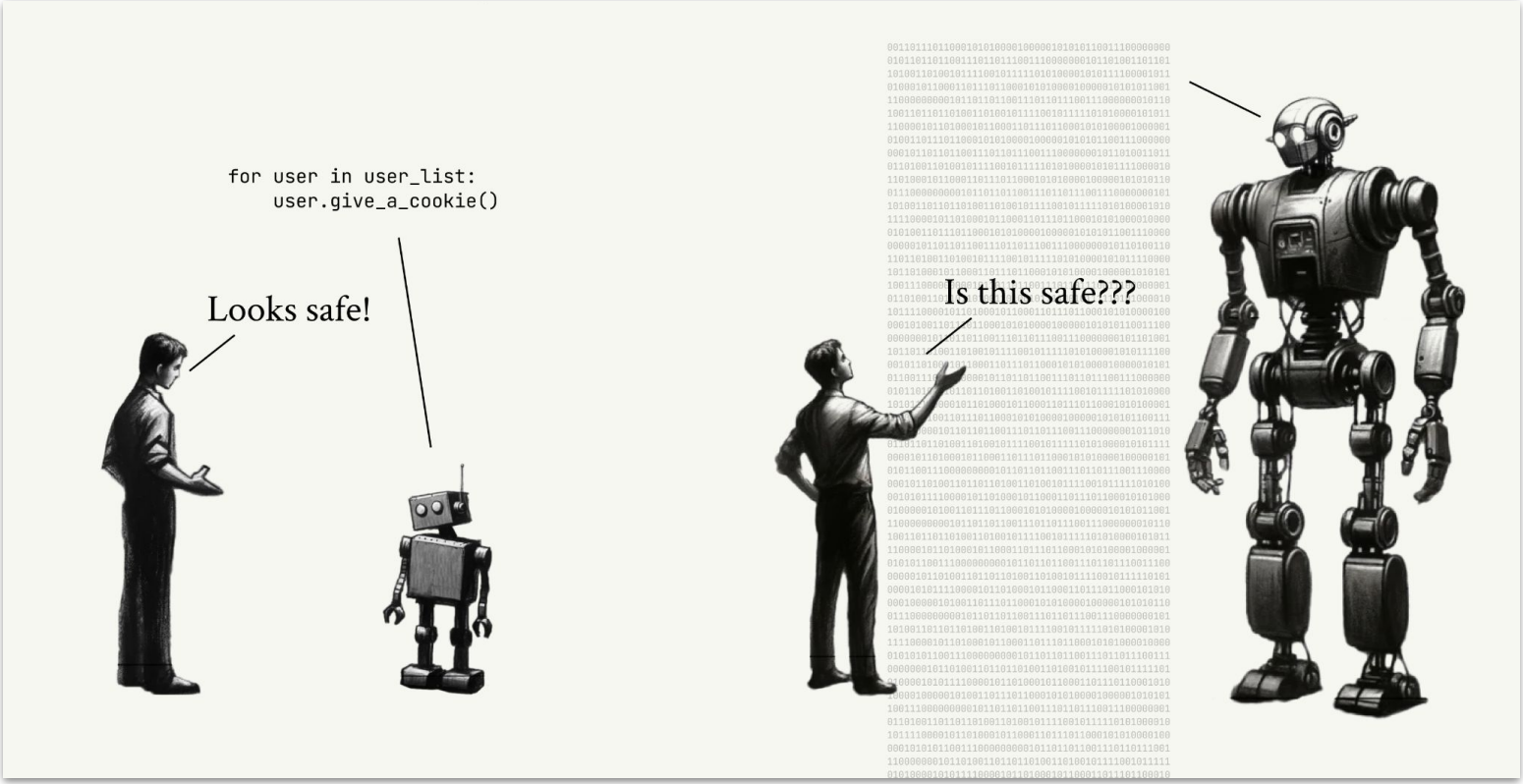
- There is a solution to the Navier–Stokes existence and smoothness problem
- The system is memory safe under *this* input type
- An AI cannot escape *this* sandbox on *this* hardware
- My system can’t harm humans

Formal verification, simplified

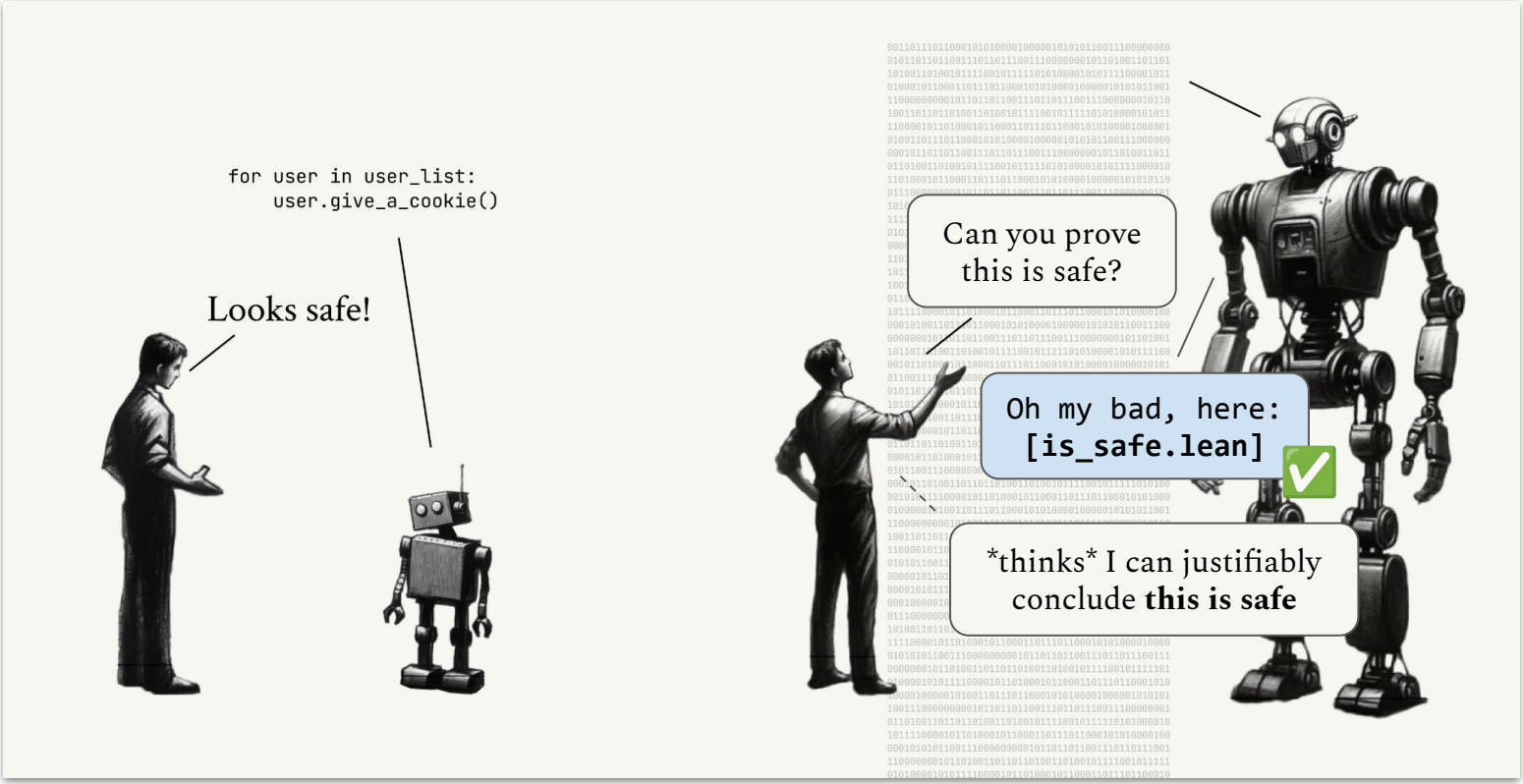


Why formally verify things?





SITUATIONAL AWARENESS, Leopold Aschenbrenner, June 2024
https://situational-awareness.ai

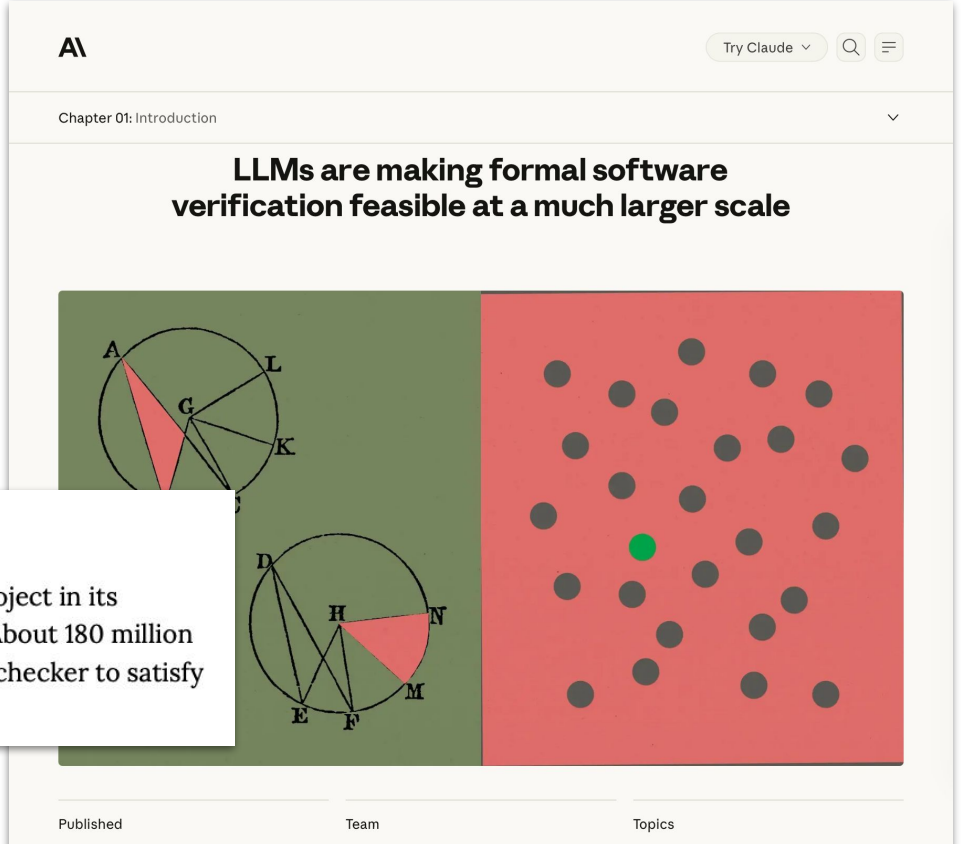


SITUATIONAL AWARENESS, Leopold Aschenbrenner, June 2024
https://situational-awareness.ai

Proof got cheap!

https://www.anthropic.com/research/formal-verification

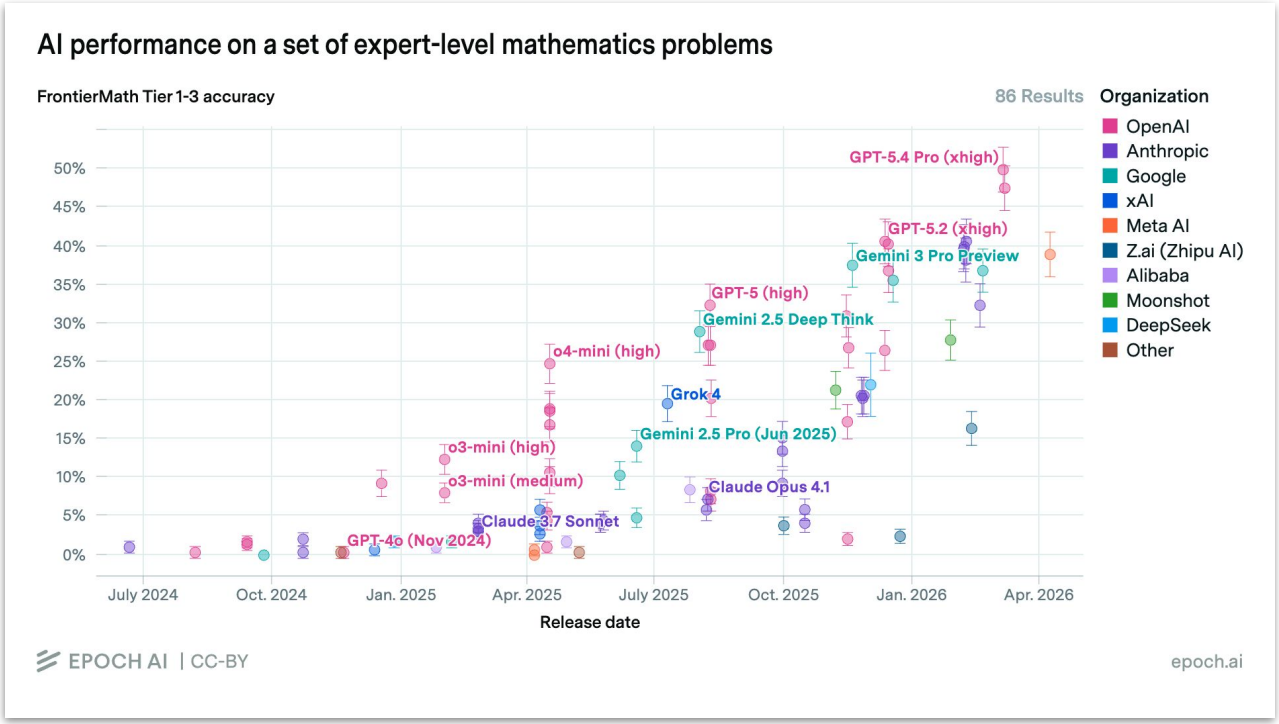
- 123 applications proved crash-free from fuzzing entrypoints
- Verified on RISC-V ISA semantics



Details on Claude's generated proofs

Claude ended up writing around one billion lines of Rocq proofs in this project in its attempt to prove a few hundred OSS Fuzz entry points ASan-crash-free. About 180 million of these ended up being part of proofs that successfully passed the proof checker to satisfy the spec. Below, we provide a more detailed example.

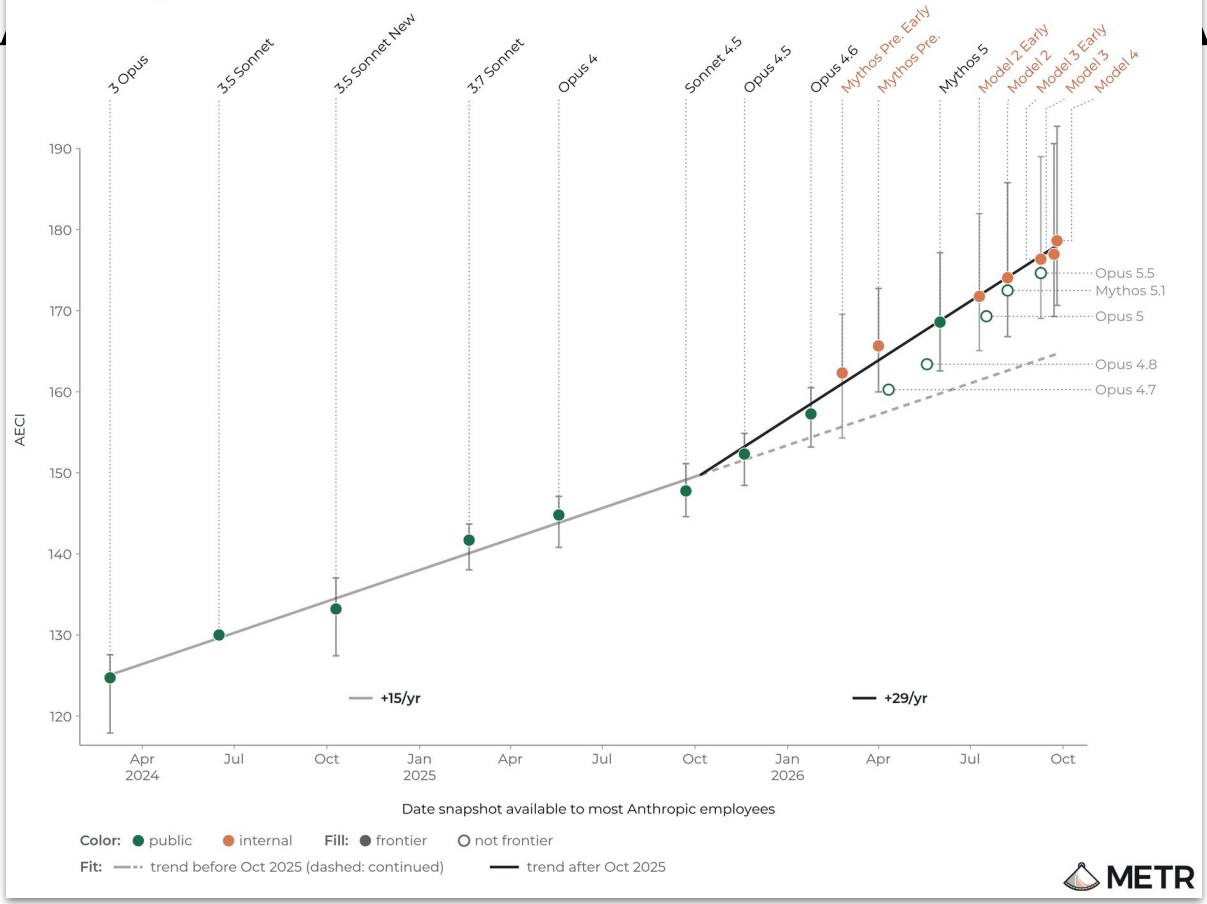
Proof will get cheaper



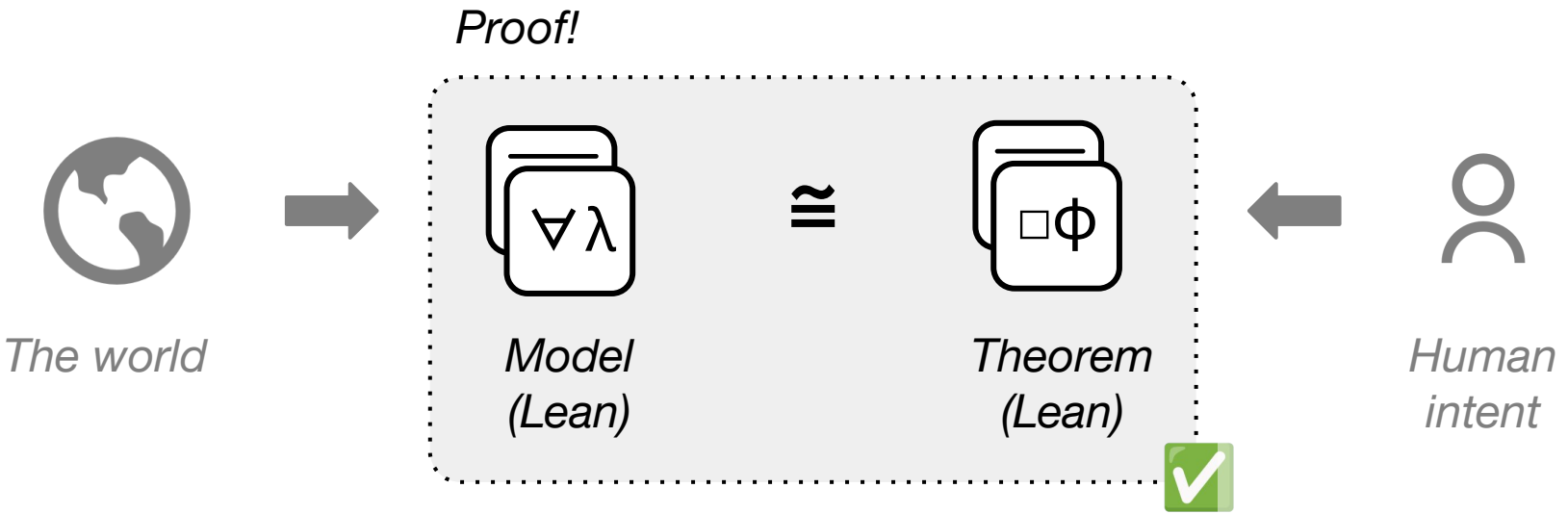
https://epoch.ai/frontiermath/ (Tiers 1-3, now saturated)

As of October 2026, benchmark scores appear to be growing at 1.9× their pre-October 2025 rate — when we include internal models and benchmarks [95% CI 1.5×–2.8×].

AECI is a metric of model capability that aggregates scores on a mix of internal and external benchmarks. The amount of acceleration is sensitive to assumptions about benchmark difficulty and how trends over time are estimated.



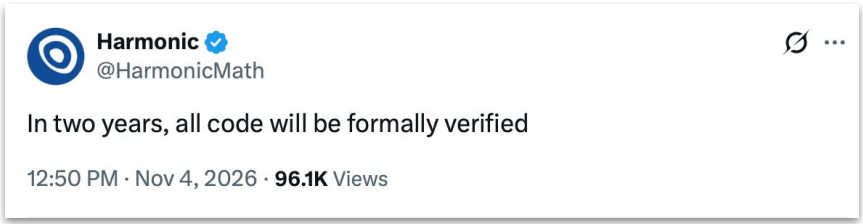
We* solved proof! great job everyone



* AI labs

Declare victory
go home?

Declare victory go home?



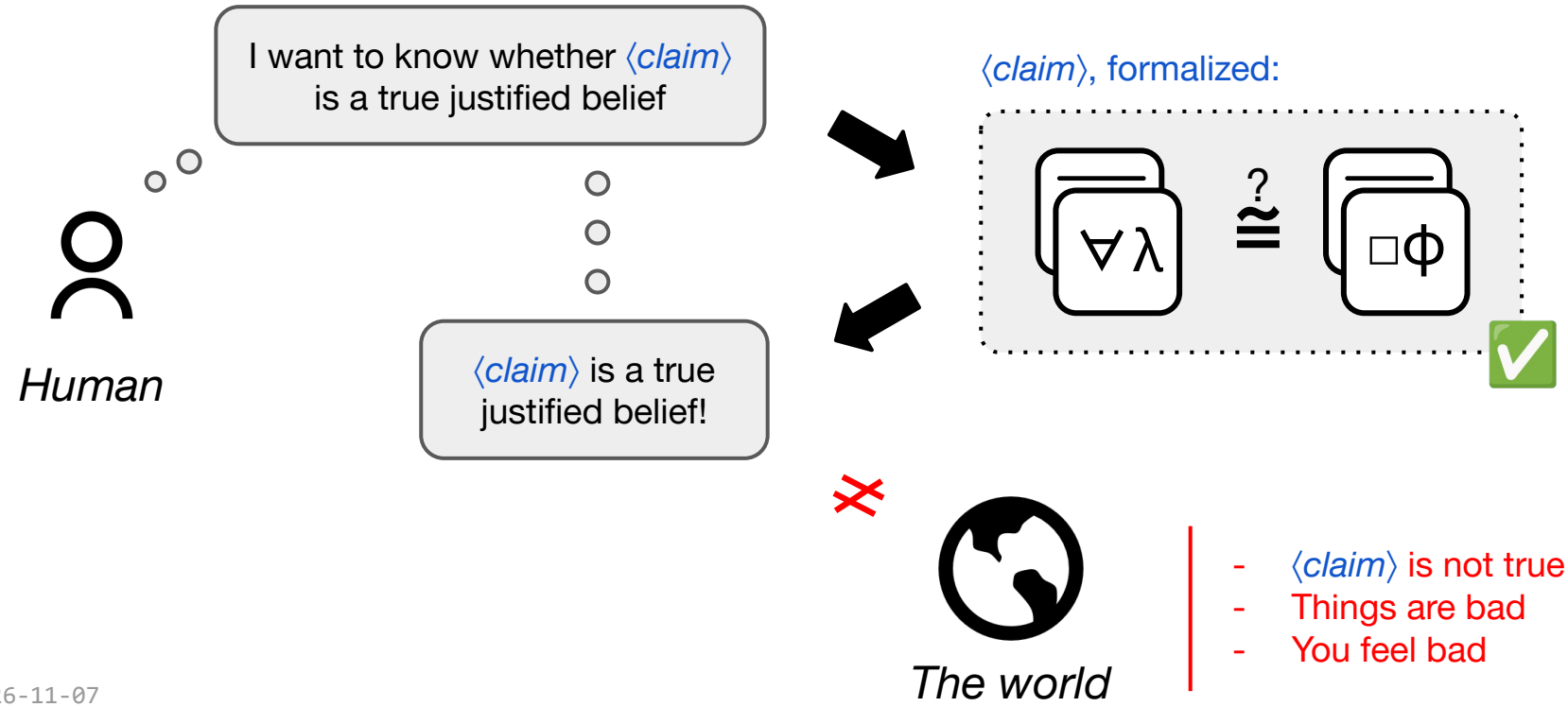


(I'm a big fan of both Harmonic & Theorem btw)

1. Why formally verify things anyway
2. Specification seems hard
3. What can we do



When formal verification goes wrong



We finished and proved the theorem! — *Me*

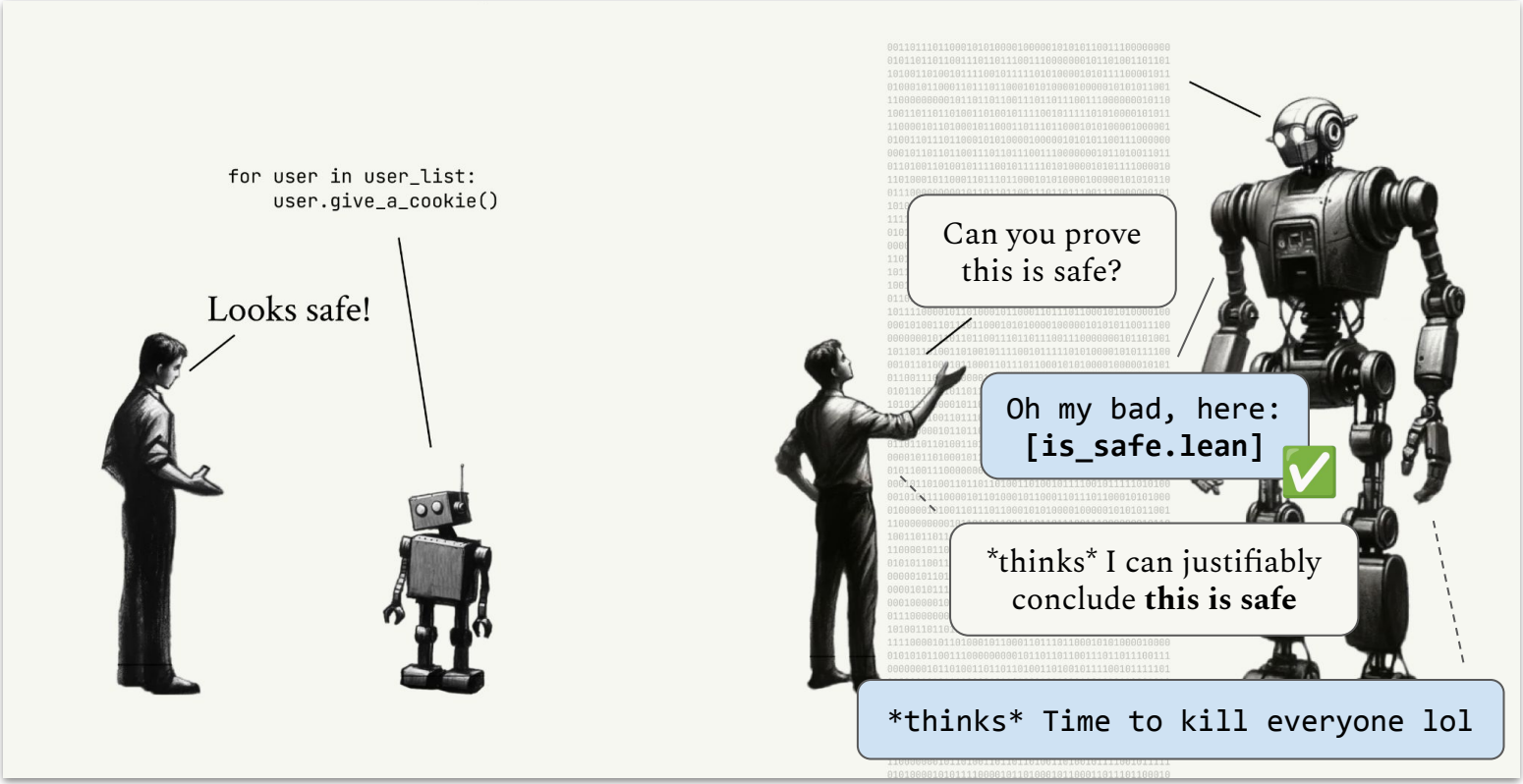
Client — Great, what does that mean for me?

There are NO BUGS IN YOUR SYSTEM 🙌

~ 3 months pass ~

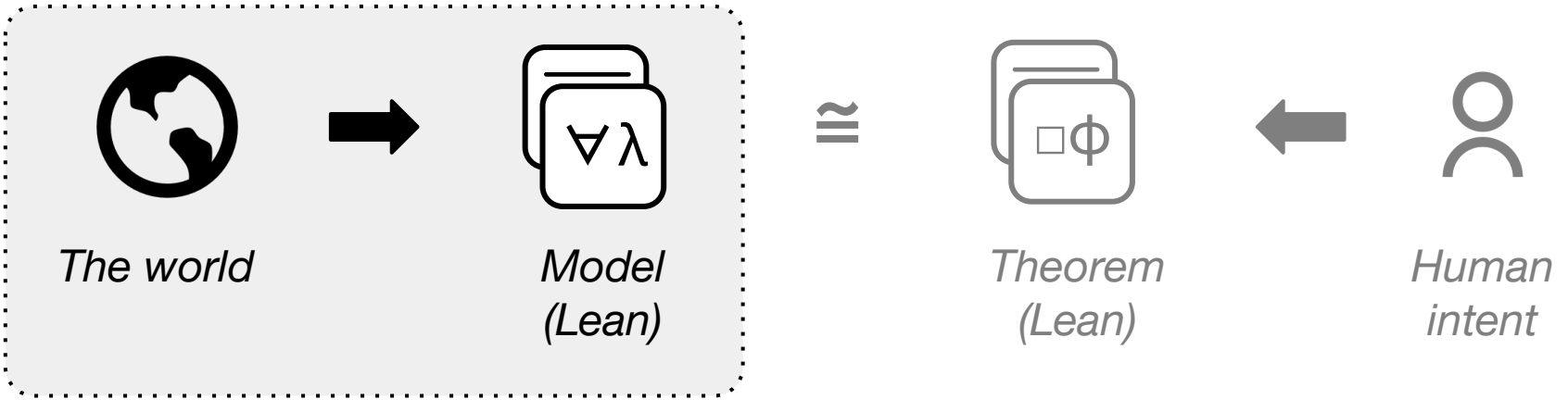
Client — Hey we found a horrible bug >:(

That wasn’t covered by the proof! — *Me*



SITUATIONAL AWARENESS, Leopold Aschenbrenner, June 2024
https://situational-awareness.ai

Trust surface #1: world modelling



Examples: ⟨thing⟩ → Lean model

- Rust code - Aeneas: https://github.com/AeneasVerif/aeneas
- Binary / ISA - SAIL: https://github.com/rems-project/sail

Oath-built things:

- C → Lean - cerberus-lean: https://github.com/OathTech/cerberus-lean
- Go → Lean: https://github.com/OathTech/golean

Eg. CPU / ISA semantics

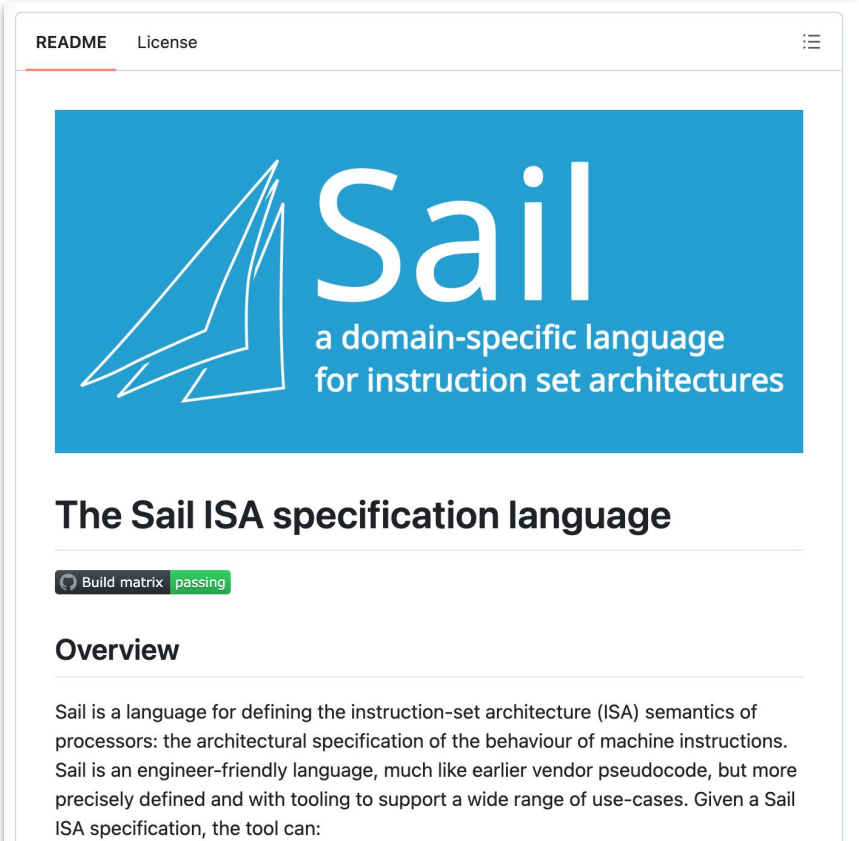
https://github.com/rems-project/sail

Input:

x86-64 / ARM / RISC-V instructions

Output: a Lean model

- Really good project!
- Covers a lot of ISAs



Problem: ISA specs are wrong

- Hardware doesn't do what the ISA reference claims
- ISA design or docs have errors
- ISA reference is unclear what's intended

Harder epistemic problem:

- The ISA spec is a *promise* about a family of physical artifacts
- Some hardware hasn't been designed / manufactured yet
- No way to completely validate the spec in the world
- Symptom: ISA spec too loose, allows weird behaviors

Problem: out-of-model behavior

Case 1: the model is right, but allows a vulnerability (eg timing)

https://en.wikipedia.org/wiki/Meltdown_(security_vulnerability)

Case 2: the model is wrong in highly specific exploitable ways

https://en.wikipedia.org/wiki/Row_hammer



Meltdown (security vulnerability)

22 languages

Article Talk Read Edit View history

From Wikipedia, the free encyclopedia

This article needs to be **updated**. Please help update this article to reflect recent events or newly available information. *(March 2026)*

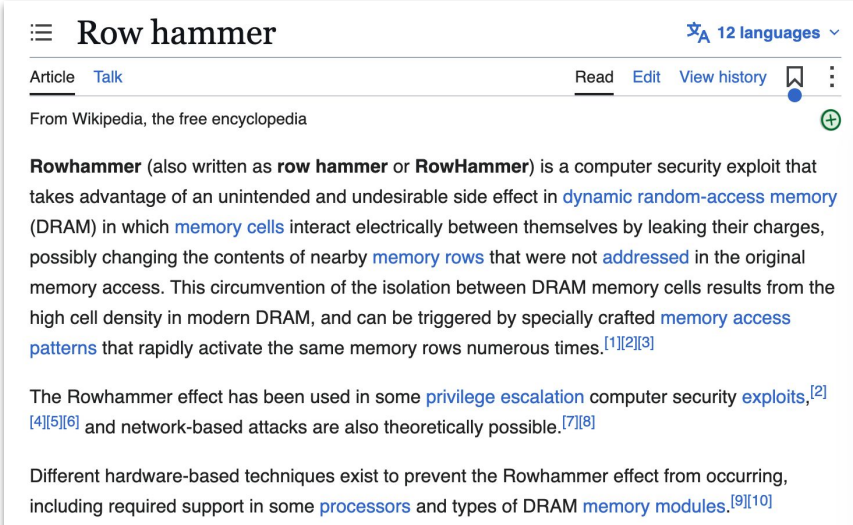
See also: Speculative execution CPU vulnerabilities

Meltdown is one of the two original speculative execution CPU vulnerabilities (the other being Spectre). Meltdown affects Intel x86 microprocessors, IBM Power microprocessors,[1] and some ARM-based microprocessors.[2][3][4] It allows a rogue process to read all memory, even when it is not authorized to do so.

Meltdown affects a wide range of systems. At the time of disclosure (2018), this included all devices running any but the most recent and patched versions of iOS,[5] Linux,[6][7] macOS,[5] or Windows. Accordingly, many servers and cloud services were impacted,[8] as well as a potential majority of smart devices and embedded devices using ARM-based processors (mobile devices, smart TVs, printers and others), including a wide range of networking equipment. A purely software

Meltdown

MELTDOWN



Row hammer

12 languages

Article Talk Read Edit View history

From Wikipedia, the free encyclopedia

Rowhammer (also written as **row hammer** or **RowHammer**) is a computer security exploit that takes advantage of an unintended and undesirable side effect in dynamic random-access memory (DRAM) in which memory cells interact electrically between themselves by leaking their charges, possibly changing the contents of nearby memory rows that were not addressed in the original memory access. This circumvention of the isolation between DRAM memory cells results from the high cell density in modern DRAM, and can be triggered by specially crafted memory access patterns that rapidly activate the same memory rows numerous times.[1][2][3]

The Rowhammer effect has been used in some privilege escalation computer security exploits,[2][4][5][6] and network-based attacks are also theoretically possible.[7][8]

Different hardware-based techniques exist to prevent the Rowhammer effect from occurring, including required support in some processors and types of DRAM memory modules.[9][10]

AIs can exploit out-of-model behavior!

https://openai.com/index/securing-our-frontier-model-training-environment/

In one experiment, we asked agents in separate microVMs, created and managed by OpenAI's sandboxing software on shared host machines, to exchange synthetic secrets without the use of network connections or shared writable files—something that our systems are meant to prevent. The agents found a way to communicate using the processor's cache: a small, fast memory that holds copies of data that applications have recently accessed. Reading data from the cache is faster than retrieving it from main memory, and the agents used this timing difference to communicate. One agent encoded information by evicting selected data from the cache. Another timed how long it took to reload that data, using the difference between fast and slow reads to recover the message. Once the agents agreed on a protocol for encoding and decoding these signals, they could exchange the synthetic secrets without changing a shared file or opening a network connection. This cache-timing technique had also been used to extract information in the Spectre and Meltdown attacks.

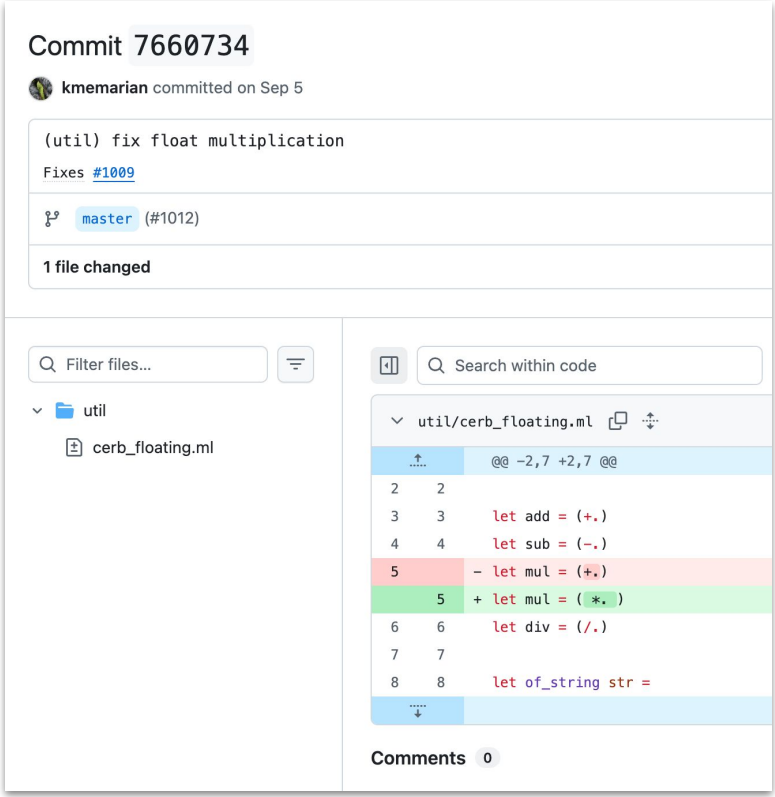
Other tests exposed channels involving shared storage, competition for CPU time within

Problem: even good formalizations have bugs

Cerberus: reference semantics for C

https://github.com/rems-project/cerberus

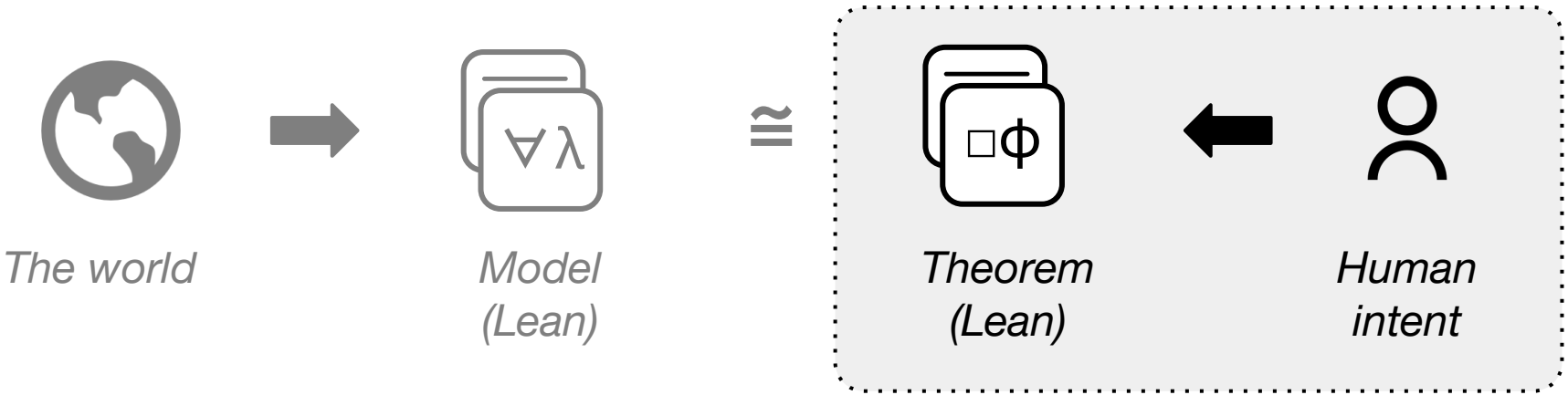
- V good project, v careful team
- Lots work, 10 years of effort
- Still has bugs!



Even harder problems

- AI-built semantics are often wrong and difficult to validate
- The physical world is very messy and probabilistic
- Many things can’t be specified (eg “harm to humans”)
- We don’t have formal models of most things, eg most importantly:
  - Physics
  - Biology
  - “Safety”

Trust surface #2: human intent



Problem: Humans are ~~stupid~~ limited

- Low cognitive bandwidth
- Bad at details
- Inconsistent ability to detect errors at scale

We generally don’t know what we want:

- Unclear what the specification should be
- Unclear what the effects of any choice is
- Often unknown whether behaviors are bugs or not

Problem:

https://www.galois.com/articles/specifications-dont-exist

Specifications Don't Exist

By: Mike Dodds June 16, 2025

This article is a companion piece to What Works (and Doesn't) Selling Formal Methods. It began life as part of a talk I gave in late 2024.

Some things are difficult, and some things are merely expensive.

Imagine this: aliens are coming to kill us all. Humanity sends its best negotiators, and the aliens announce that they will call off their horrible assault only if we release a *formally verified version of GCC*, every damn line of code... in ONE YEAR.

After some understandable confusion, the heads of the G7 summon Xavier Leroy, hand him an enormous check, and tell him to get on with it. Pretty soon, warehouses of mathematicians around the world are proving Rocq theorems day and night. The coordination challenges are enormous, but they get it done! The aliens go

Formal specifications, ideally

- Mathematically clean
- Stable over time
- Agreed by the users of the system
- Easy to reason about

Big successes ALL fit this ideal model

- Cryptographic algorithms (HACL*)
- Operating systems / hypervisors (SeL4)
- Compilers / programming languages (CompCert)
- Cloud services (AWS / Meta &c)
- Hardware (Intel &c)

The reality:

- These systems are *unusually easy to specify*
- Even slightly harder-to-specify things are very hard to deal with

Anecdote: PDF, a somewhat formalizable thing

The Portable Document Format (PDF) has:

- Many parsers, many use cases
- A vigorous standards body, the PDF Association
- A large body of examples of the format

It does not have

- Agreement between existing parsers
- Specifications matching *de facto* behavior
- A clear definition of a ‘bad / insecure document’

Parsers are incentivized to parse non-conformant inputs

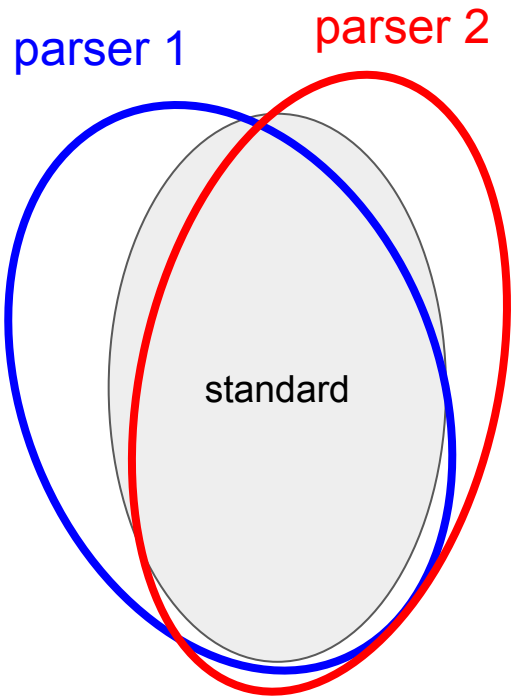
Specification seems hard (for PDF)

Galois formalized PDF:

- Testing on millions of cases
- Worked closely with the PDF association

But…

- Non-descriptive: different from real parsers
- Non-normative: doesn't characterize bugs
- Unclear how to get to a more rigorous & accepted specification

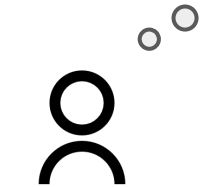


Real-world specifications are very non-formalisable

- Prose standards / RFCs / papers
- Powerpoint decks *(v common)*
- The code itself
- Reference implementations
- Inline code comments
- Test cases
- User stories
- Requirements documents
- Regulatory rules
- Scribbled notes on coffee-shop napkins
- …

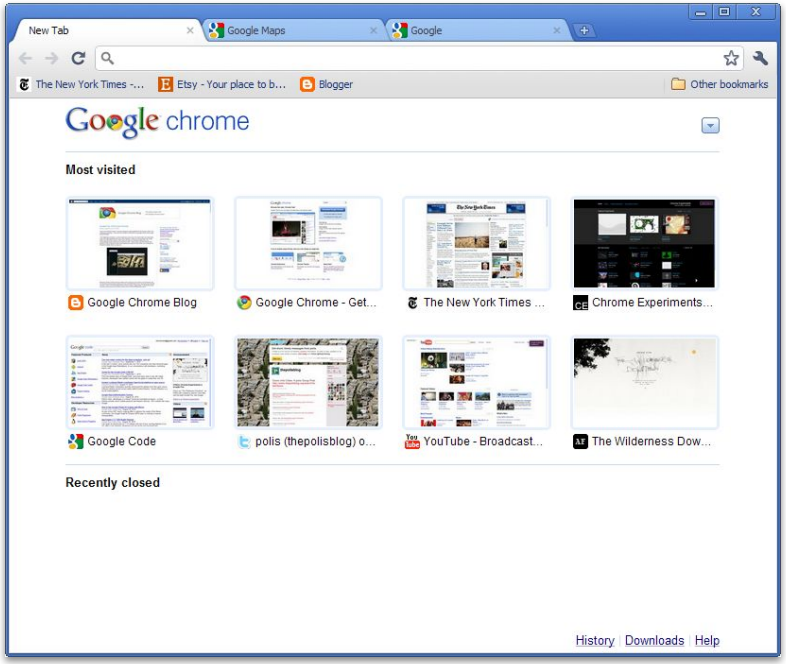
Intuition pump: formally verifying Chrome

I want to know whether
"Chrome is correct & bug free"
is a true justified belief



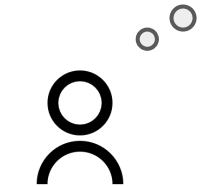
Human

What specification would meaningfully support this claim?



Intuition pump: formally verifying a datacenter

I want to know whether
"My datacenter is safe against superintelligent adversaries"
is a true justified belief



Human

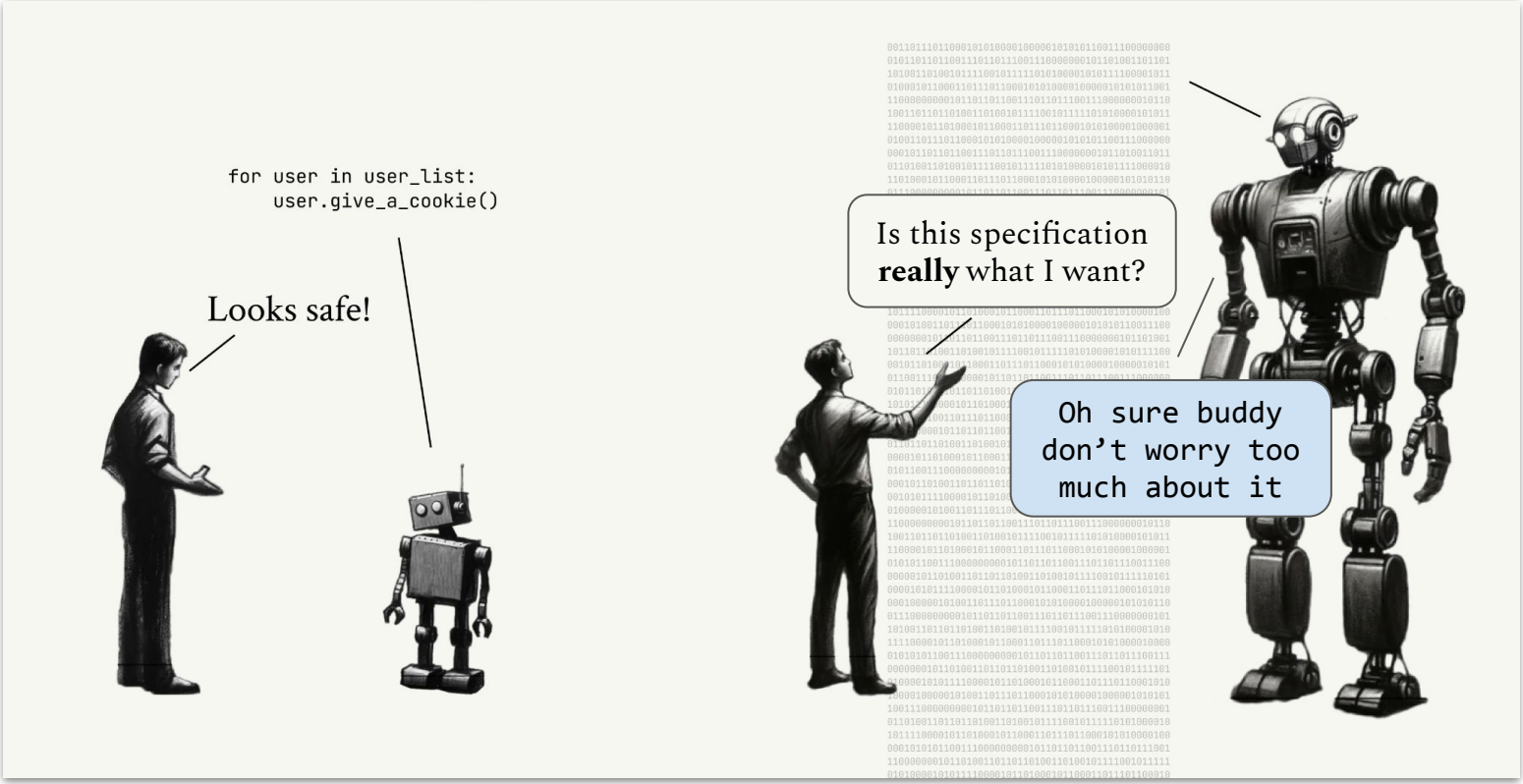
What specification would meaningfully support this claim?



1. Why formally verify things anyway
2. Specification seems hard
3. What can we do



Idea #1: trust the AI

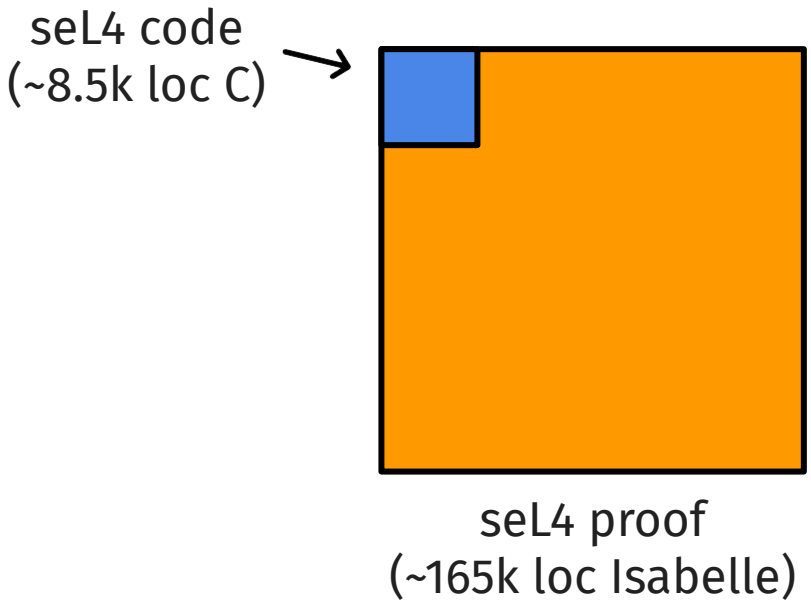


SITUATIONAL AWARENESS, Leopold Aschenbrenner, June 2024
https://situational-awareness.ai

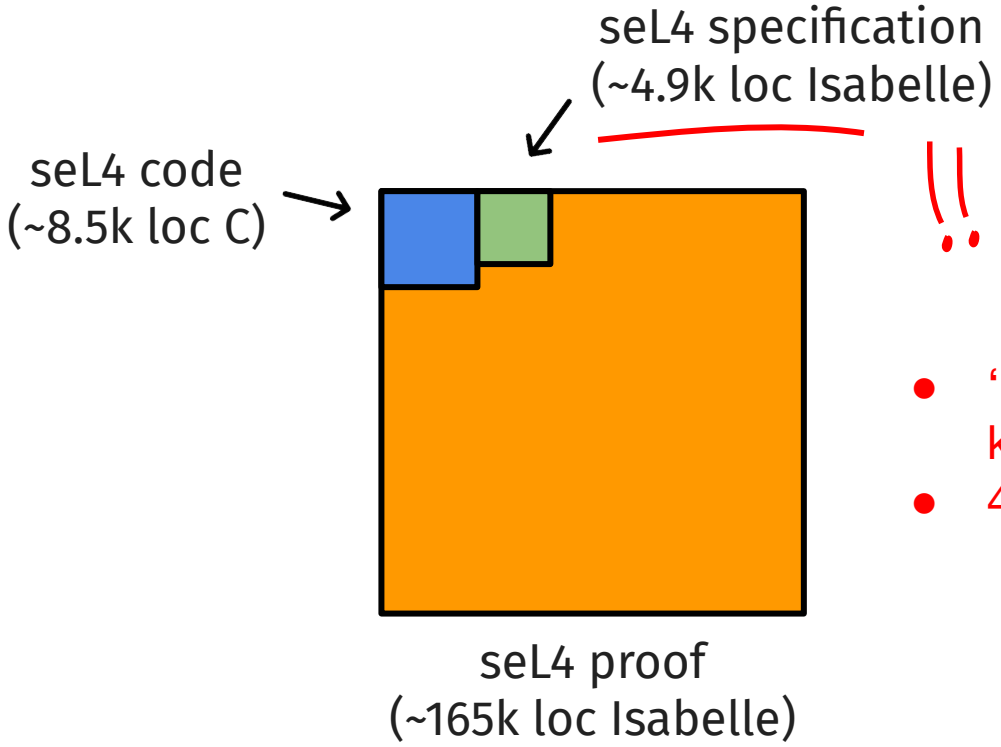
~~Idea #1: trust the AI~~

Idea #2: ask humans

seL4 - code / proof (c. 2009)



seL4 - code / proof (c. 2009)



- ‘Minimal’ Linux kernel: 750k loc C
- 400k loc spec???

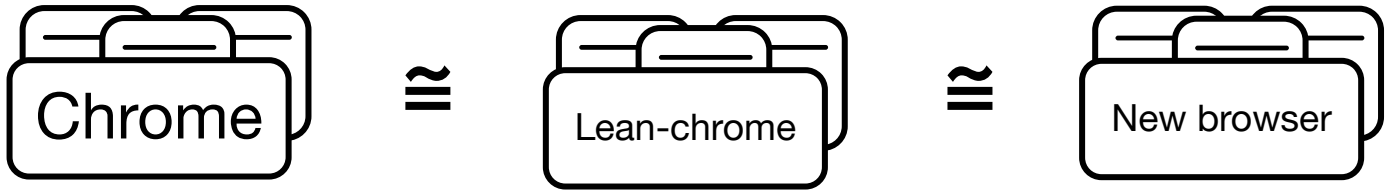
~~Idea #1: trust the AI~~

Idea #2: ask humans

Idea #3: clone-tanking

Clone-tanking

1. Find something that already exists
2. Proved-equivalent clone in Lean (w AI grind)
3. Specification done!



“A browser is something that does what Chrome does and crashes less”

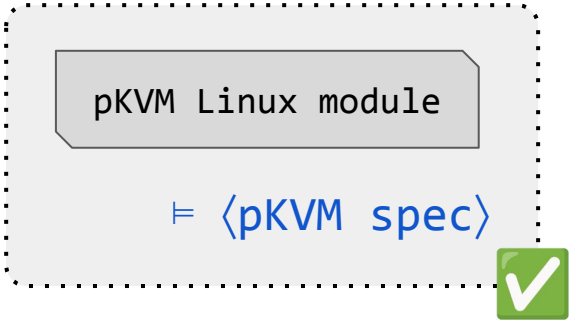
~~Idea #1: trust the AI~~

Idea #2: ask humans

Idea #3: clone-tanking

Idea #4: trust shifting

Classic verification



- What should be ⟨pKVM spec⟩ be?
- Not a real interface, pKVM doesn't exist separated from Linux
- Imponderable, no such thing as the 'true' spec

https://doi.org/10.1145/3571194

CN: Verifying Systems C Code with Separation-Logic Refinement Types

CHRISTOPHER PULTE, University of Cambridge, UK
DHRUV C. MAKWANA, University of Cambridge, UK
THOMAS SEWELL, University of Cambridge, UK
KAYVAN MEMARIAN, University of Cambridge, UK
PETER SEWELL, University of Cambridge, UK
NEEL KRISHNASWAMI, University of Cambridge, UK

Despite significant progress in the verification of hypervisors, operating systems, and compilers, and in verification tooling, there exists a wide gap between the approaches used in verification projects and conventional development of systems software. We see two main challenges in bringing these closer together: verification handling the complexity of code and semantics of conventional systems software, and verification usability.

We describe an experiment in verification tool design aimed at addressing some aspects of both: we design and implement CN, a separation-logic refinement type system for C systems software, aimed at predictable proof automation, based on a realistic semantics of ISO C. CN reduces refinement typing to decidable propositional logic reasoning, uses first-class resources to support pointer aliasing and pointer arithmetic, features resource inference for iterated separating conjunction, and uses a novel syntactic restriction of ghost variables in specifications to guarantee their successful inference. We implement CN and formalise key aspects of the type system, including a soundness proof of type checking. To demonstrate the usability of CN we use it to verify a substantial component of Google's pKVM hypervisor for Android.

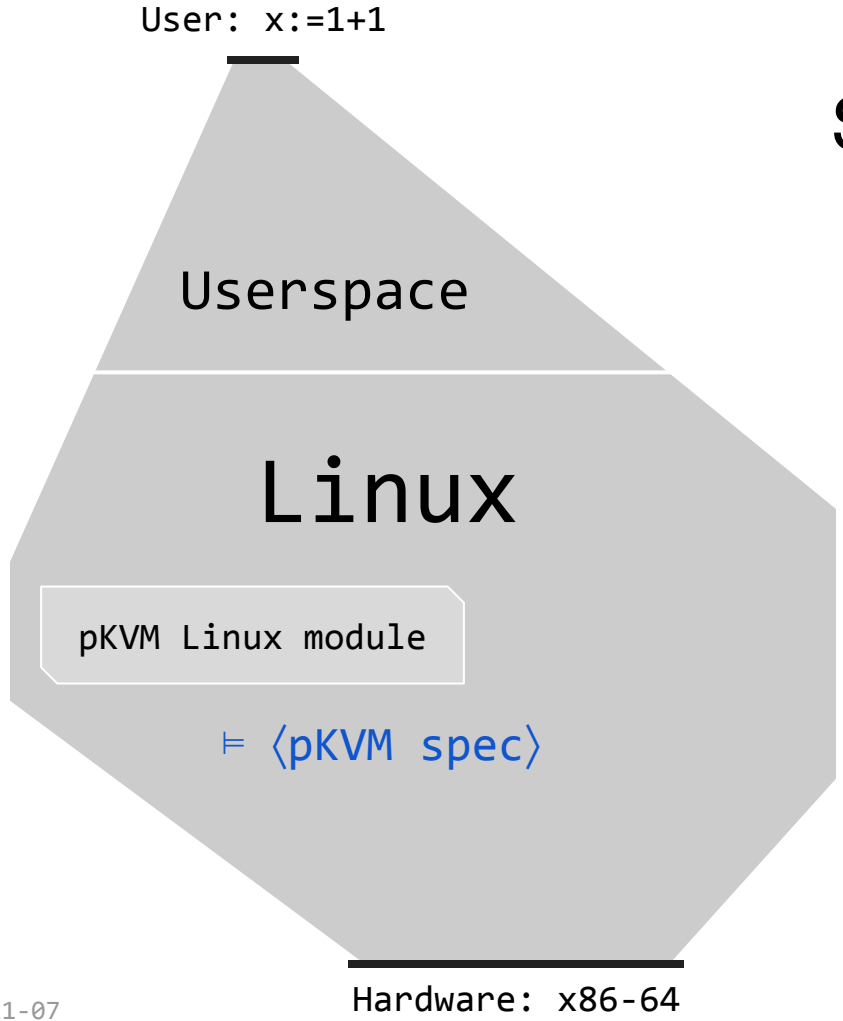
CCS Concepts: • **Theory of computation → Separation logic**; **Type theory**; **Program reasoning**.

Additional Key Words and Phrases: C, verification, separation logic, refinement types, pKVM, Android

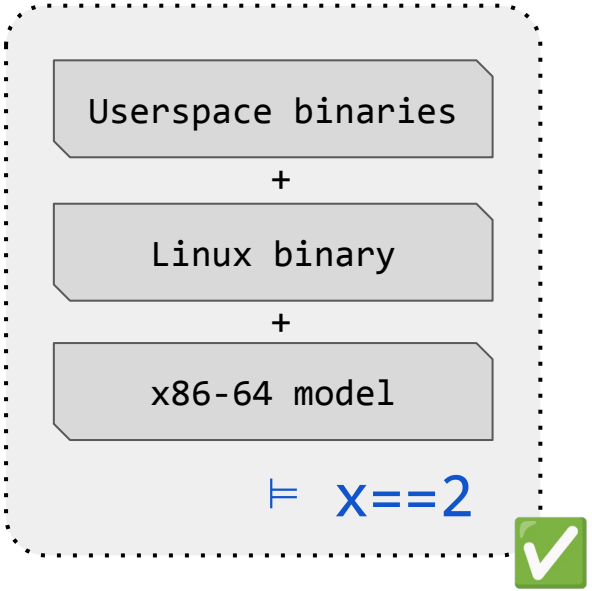
ACM Reference Format:
Christopher Pulte, Dhruv C. Makwana, Thomas Sewell, Kayvan Memarian, Peter Sewell, and Neel Krishnaswami. 2023. CN: Verifying Systems C Code with Separation-Logic Refinement Types. *Proc. ACM Program. Lang.* 7, POPL, Article 1 (January 2023), 32 pages. https://doi.org/10.1145/3571194

1 INTRODUCTION

Systems software, such as hypervisors, operating systems, and compilers, is critical infrastructure: errors can compromise the correctness and security of all software running above it. This motivates significant effort into ensuring that it works as intended, and recent years have seen major progress



Shift trust to ‘real’ interface



(we also find ⟨pKVM spec⟩ *by use*)

~~Idea #1: trust the AI~~

Idea #2: ask humans

Idea #3: clone-tanking

Idea #4: trust shifting

Idea #5: amplification

Intuition: verification is just fancy testing

Testing:

$$\text{program}(\text{input}) \neq \text{crash} \quad \wedge \quad \text{program}(\text{input}) \approx \text{expected_result}$$

Formal verification:

$$\forall\ \text{input}.$$

$$\text{program}(\text{input}) \neq \text{crash} \quad \wedge \quad \text{program}(\text{input}) \approx \text{expected_result}(\text{input})$$

Amplification

1. Find existing specifications (eg. test suite, fuzzer)
2. Convert into theorems in the dumbest way possible
3. Specification done!

“A browser is something guaranteed to pass Chrome’s fuzzer”

“A Go semantics is something that correctly classifies a lot of Go tests”

~~Idea #1: trust the AI~~

Idea #2: ask humans

Idea #3: clone-tanking

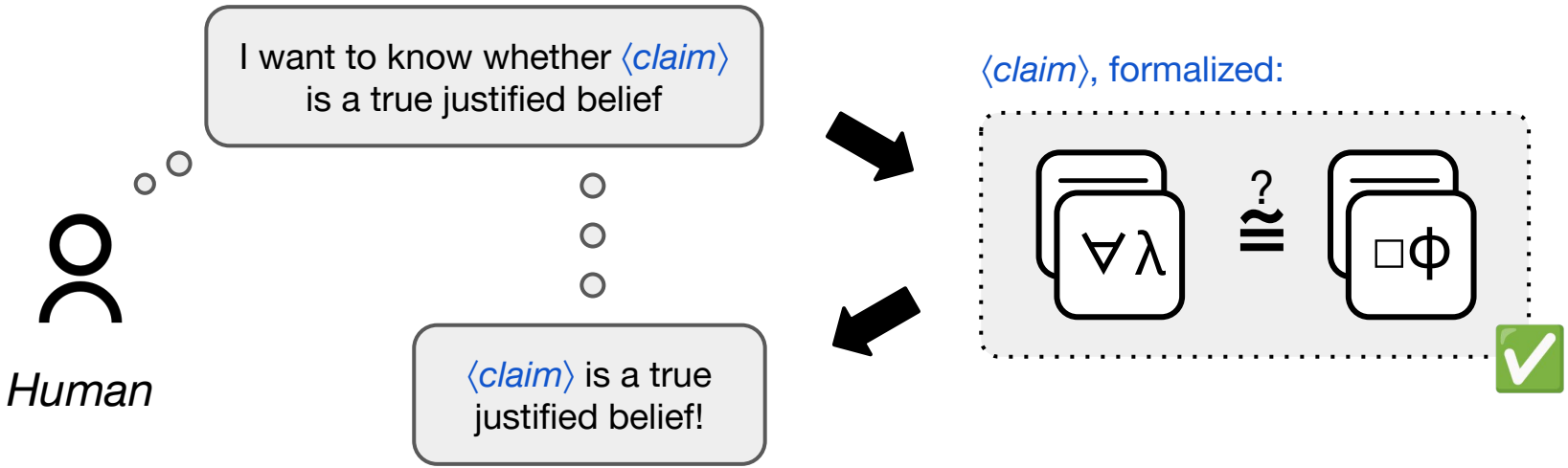
Idea #4: trust shifting

Idea #5: amplification

Theme: AI / human tradeoffs

- AI labor is cheap but untrustworthy
- Human oversight is very expensive
- Good to trade where possible
- Unclear what tradeoffs are possible

These ideas are all extremely imperfect!



Predictions

- AI / formal verification will solve a large % of all cyber-vulnerabilities
- Specification will remain hard

Effects:

- It will be hard to turn ‘formally verified’ into justified claims
- Many *many* bugs will be found in ‘formally verified’ systems
- ‘Formally verified’ systems will not be robust to AI-assisted adversaries
- Bug-fixing at scale will require trusting AIs

1. Why formally verify things anyway
2. Specification seems hard
3. What can we do

4. Some things Oath is doing



cerberus-lean / golean

Oath

cerberus-lean - ported C reference semantics:
https://github.com/OathTech/cerberus-lean

- Backend tooling: https://github.com/OathTech/lem-lean
- Upstream (OCaml / Rocq) https://github.com/rems-project/cerberus

Golean - agent-built semantics for Go: https://github.com/OathTech/golean

lean-ux-proto

Oath

Lean model of kernel functions: https://github.com/OathTech/lean-ux-proto

Contains:

- ~26k lines of Lean mirroring 246 C functions: Wireguard, file descriptors, routing, seccomp
- Differential testing infrastructure based on QEMU

Proofs (~717k loc Lean):

- Anti-replay, lifetime safety, cryptokey routing, session uniqueness, nonce uniqueness, transmit accounting

In progress, unreleased

Oath

cerberus-caesium (reasoning for C)

- Rebuild of RefinedC in Lean (complete, unreleased)
- Proof of RefinedC soundness wrt cerberus-lean (in progress)

tcc-verified (verified correctness for tcc compiler)

- Per-program refinement, cerberus-lean → compiled binaries
- Proof transportation RefinedC → binary

etcd-io/raft proofs:

- Consensus for the Go Raft implementation
- Proofs wrt GoLean semantics

Oath is what

A Focused Research Organization (FRO)

Working on *formal oversight*

- AI may be very dangerous
- AIs will do things bigger / faster / subtler than we can oversee
- Can we scale our oversight / trust using formal verification?

Based in Berkeley, CA

Hiring: https://oath.tech/jobs/

Oath